

ACM-XCPC

slmhh

代码模版库



湖北工业大学
HUBEI UNIVERSITY OF TECHNOLOGY

Template For XCPC

slmhh @ HBUT

August 22, 2026

Contents

1 字符串	3
1.1 字符串哈希	3
1.2 最小表示法	3
1.3 KMP	3
1.4 Z 函数 (扩展 KMP)	4
1.5 Manacher	4
1.6 字典树	4
1.7 AC 自动机	5
1.8 后缀数组	5
1.9 回文自动机 PAM(回文树)	6
1.10 后缀自动机 (SAM)	6
1.10.1 后缀自动机应用	7

1 字符串

1.1 字符串哈希

$\mathcal{O}(|S|)$

```

1  typedef std::pair<i64,i64> PII;
2  const int N = 1e6 + 10;
3
4  i64 pow1[N],pow2[N];
5  struct string_hash{
6      const int Base = 499,M1 = 998244853,M2 = 989244353;
7      std::string str;
8      std::vector<i64> hash1,hash2;
9      void extend(char c,int idx){
10         pow1[idx] = (pow1[idx - 1] * Base) % M1;
11         hash1[idx] = (hash1[idx - 1] * Base + c) % M1;
12         pow2[idx] = (pow2[idx - 1] * Base) % M2;
13         hash2[idx] = (hash2[idx - 1] * Base + c) % M2;
14     }
15
16     void init(int n){
17         hash1.resize(n + 1,0);
18         hash2.resize(n + 1,0);
19         for(int i = 0;i <= n + 2;i++){
20             pow1[i] = pow2[i] = 1;
21         }
22
23         void init(std::string str){
24             i64 n = str.size();
25             init(n);
26             this->str = str;
27             for(int i = 0;i < n;i++){
28                 extend(str[i],i + 1);
29             }
30         }
31     string_hash(){}
32     string_hash(std::string str){init(str);}
33
34     PII getHash(i64 l,i64 r){ // 1_idx
35         i64 t1 = (hash1[r] - 1LL * hash1[l - 1] * pow1[r - l + 1]) % M1;
36         t1 = (t1 + M1) % M1;
37         i64 t2 = (hash2[r] - 1LL * hash2[l - 1] * pow2[r - l + 1]) % M2;
38         t2 = (t2 + M2) % M2;
39         return {t1,t2};
40     }
41
42     bool same(i64 l1,i64 r1,i64 l2,i64 r2){
43         PII ta = getHash(l1,r1);
44         PII tb = getHash(l2,r2);
45         return ta.first == tb.first && ta.second == tb.second;
46     }
47 };

```

1.2 最小表示法

最小表示法是一种用于解决字符串最小表示问题的方法。它的核心思想是找到一个字符串的所有循环同构字符串中字典序最小的那个字符串, $\mathcal{O}(|S|)$ 。

```

1  template<typename T>
2  int get_min_str(const T& str){ //返回循环同构字符串的最小字典序首位下标
3      int i = 0,j = 1,k = 0;
4      int n = str.size();
5      while(i < n && j < n && k < n){
6          int tmp = str[(i + k) % n] - str[(j + k) % n];
7          if(tmp == 0){
8              k++;
9          }
10         else{
11             if(tmp > 0) i += k + 1;
12             else j += k + 1;
13             if(i == j) j++;
14             k = 0;
15         }
16     }
17     return std::min(i,j);
18 }

```

1.3 KMP

在字符串中查找子串, 以最坏 $\mathcal{O}(N + M)$ 的时间计算 t 在 s 中出现的全部位置。

最小周期: 字符串长度-整个字符串的 *border*;

最小循环节: 区别于周期, 当字符串长度 $n \bmod (n - \text{next}[n]) = 0$ 时, 等于最小周期, 否则为 n 。

```

1  i64 ne[N],m,n;
2  std::string a,b;
3  std::vector<i64> ans;
4  void getNext(std::string& str){
5      ne[0] = 0;
6      for(int i = 1,j = 0;i < str.size();i++){
7          while(j && str[i] != str[j]) j = ne[j - 1];
8          if(str[i] == str[j]) j++;
9          ne[i] = j;
10     }
11 }
12
13 void kmp(std::string& a,std::string& b){
14     getNext(b);
15     for(int i = 0,j = 0;i < a.size();i++){
16         while(j && a[i] != b[j]) j = ne[j - 1];
17         if(a[i] == b[j]) j++;
18         if(j == b.size()) ans.push_back(i - b.size() + 2);

```

```

19 }
20 }

```

1.4 Z 函数 (扩展 KMP)

获取字符串 s 和 $s[i, n-1]$ (即以 $s[i]$ 开头的后缀) 的最长公共前缀 (LCP) 的长度, 总复杂度 $\mathcal{O}(N)$ 。

```

1 template<typename T>
2 std::vector<int> z_function(const T& str){
3     int n = str.size();
4     std::vector<int> z(n);
5     int j = 0;
6     for(int i = 1; i < n; i++){
7         z[i] = (j + z[j] <= i) ? 0 : std::min(j + z[j] - i, z[i - j]);
8         while(i + z[i] < n && str[z[i]] == str[i + z[i]]){
9             z[i]++;
10        }
11        if(j + z[j] < i + z[i]){
12            j = i;
13        }
14    }
15    z[0] = n;
16    return z;
17 }

```

1.5 Manacher

$\mathcal{O}(|S|)$

```

1 struct Manacher{
2     std::vector<int> p;
3     std::string change(std::string& str){
4         std::string temp = "!#";
5         for(int i = 0; i < str.size(); i++){
6             temp += str[i];
7             temp += '#';
8         }
9         return temp;
10    }
11
12    void get_p(std::string& str){
13        int maxright = 0, mid = 0;
14        str = change(str);
15        p.assign(str.size(), 0);
16        for(int i = 1; i < str.size(); i++){
17            if(i < maxright){
18                p[i] = std::min(p[2 * mid - i], maxright - i + 1);
19            }
20            else{
21                p[i] = 1 + (str[i] != '#');
22            }

```

```

23         while(i - p[i] >= 0 && p[i] + i < str.size() && str[i - p[i]] == str[i + p[i]]){
24             p[i]++;
25         }
26         if(i + p[i] - 1 > maxright){
27             mid = i;
28             maxright = i + p[i] - 1;
29         }
30     }
31 }
32 // 返回最大回文串长度
33 int get_max(std::string& str){
34     get_p(str);
35     int res = 0;
36     for(int i = 1; i < p.size(); i++){
37         res = std::max(res, p[i] - 1);
38     }
39     return res;
40 }
41 };

```

1.6 字典树

$\mathcal{O}(N \log N)$

```

1 template<int N = 26>
2 struct Trie{
3     std::vector<std::array<int, N>> son;
4     std::vector<int> cnt;
5
6     Trie(){
7         son.push_back(std::array<int, N>{});
8         cnt.push_back(0);
9     }
10
11    void insert(std::string& str){ //插入
12        int p = 0, n = str.size();
13        for(int i = 0; i < n; i++){
14            int ch = str[i] - '0';
15            if(!son[p][ch]){ //如果此处字母不存在就创建一个
16                son[p][ch] = son.size();
17                son.push_back(std::array<int, N>{});
18                cnt.push_back(0);
19            }
20            p = son[p][ch]; //指向下一个位置
21            // cnt[p]++;
22        }
23        cnt[p]++; //该处单词数+1
24    }
25
26    int find(std::string& str){ //找
27        int p = 0, n = str.size();
28        for(int i = 0; i < n; i++){
29            int ch = str[i] - '0';

```

```

30         if(!son[p][ch]){
31             return 0;
32         }
33         p = son[p][ch];
34     }
35     return cnt[p];
36 }
37 };

```

1.7 AC 自动机

$\mathcal{O}(\sum |s_i| + |S|)$

```

1 struct ACAM{
2     vector<int> ne,in,cnt,f;
3     vector<vector<int>> son;
4     int idx = 0;
5     ACAM(){
6         son.resize(N,vector<int>(26,0));
7         ne.resize(N,0),in.resize(N,0);
8         cnt.resize(N,0),f.resize(N,0);
9     }
10    ll insert(string str){ //插入模式串
11        int p = 0;
12        for(int i = 0;str[i];i++){
13            int a = str[i] - 'a';
14            if(!son[p][a]) son[p][a] = ++idx; //如果此处字母不存在就创建一个
15            p = son[p][a]; //指向下一个位置
16        }
17        cnt[p]++; //该处单词数+1
18        return p;
19    }
20
21    void build(){
22        queue<int> q;
23        for(int i = 0;i < 26;i++){
24            if(son[0][i]) q.push(son[0][i]);
25        }
26        while(!q.empty()){
27            int u = q.front();
28            q.pop();
29            for(int i = 0;i < 26;i++){
30                int v = son[u][i];
31                if(v){
32                    ne[v] = son[ne[u]][i]; //建立回跳边
33                    q.push(v);
34                    in[son[ne[u]][i]]++;
35                }
36                else son[u][i] = son[ne[u]][i]; //建转移边
37            }
38        }
39    }
40    void query(string str,int n){
41        for(int k = 0,i = 0;k < str.size();k++){

```

```

41        i = son[i][str[k] - 'a'];
42        f[i]++;
43        // for(int j = i;j && ~cnt[j];j = ne[j]){
44        //     ans += cnt[j];
45        //     cnt[j] = -1; //不重复计数
46        // }
47    }
48    queue<ll> q;
49    for(int i = 1;i <= idx;i++){
50        if(in[i] == 0) q.push(i);
51    }
52    while(!q.empty()){
53        ll u = q.front(),v = ne[u];
54        q.pop();
55        f[v] += f[u],in[v]--;
56        if(v != 0 && in[v] == 0) q.push(v);
57        f[u] *= cnt[u];
58    }
59 };

```

1.8 后缀数组

把字符串的所有后缀按字典序排序后得到的 rk 数组, 以 $\mathcal{O}(N \log N)$ 的复杂度求解。

```

1 struct SA{
2     std::string str;
3     int n,m,tp = 128,p = 0;
4     std::vector<int> rk,sa;
5     SA(std::string _s){
6         str = " " + _s;
7         n = _s.size();
8         m = (n << 1) + 1;
9         rk.assign(m,0),sa.assign(m,0);
10        work();
11    }
12    void work(){
13        if(n == 1){
14            sa[1] = rk[1] = 1;
15            return;
16        }
17        std::vector<int> cnt(std::max(tp,n) + 1,id(n + 1));
18        for(int i = 1;i <= n;i++) cnt[rk[i] = str[i]]++;
19        for(int i = 1;i <= tp;i++) cnt[i] += cnt[i - 1];
20        for(int i = n;i >= 1;i--) sa[cnt[rk[i]]--] = i;
21
22        for(int w = 1;w < n;w <= 1,tp = p){
23            int cur = 0;
24            for(int i = n - w + 1;i <= n;i++) id[++cur] = i;
25            for(int i = 1;i <= n;i++)
26                if(sa[i] > w) id[++cur] = sa[i] - w;
27            std::fill(cnt.begin(),cnt.end(),0);
28            for(int i = 1;i <= n;i++) cnt[rk[i]]++;
29            for(int i = 1;i <= tp;i++) cnt[i] += cnt[i - 1];

```

```

30     for(int i = n; i >= 1; i--) sa[cnt[rk[id[i]]]--] = id[i];
31     std::vector<int> temp(m);
32     p = 0;
33     for(int i = 1; i <= n; i++){
34         if(rk[sa[i]] != rk[sa[i - 1]] || rk[sa[i] + w] != rk[sa[i - 1] + w]) p++;
35         temp[sa[i]] = p;
36     }
37     rk = temp;
38     if(p == n) break;
39 }
40 }
41 };

```

1.9 回文自动机 PAM(回文树)

本质不同的回文串个数: $idx - 2$; 回文子串出现次数。

对于一个字符串 s , 它的本质不同回文子串个数最多只有 $|s|$ 个, 那么, 构造 s 的回文树的时间复杂度是 $\mathcal{O}(|s|)$ 。

```

1 struct PalindromeAutomaton {
2     constexpr static int N = 5e5 + 10;
3     int tr[N][26], fail[N], len[N];
4     int cntNodes, last;
5     int cnt[N];
6     string s;
7     PalindromeAutomaton(string s) {
8         memset(tr, 0, sizeof tr);
9         memset(fail, 0, sizeof fail);
10        len[0] = 0, fail[0] = 1;
11        len[1] = -1, fail[1] = 0;
12        cntNodes = 1;
13        last = 0;
14        this->s = s;
15    }
16    void insert(char c, int i) {
17        int u = get_fail(last, i);
18        if (!tr[u][c - 'a']) {
19            int v = ++cntNodes;
20            fail[v] = tr[get_fail(fail[u], i)][c - 'a'];
21            tr[u][c - 'a'] = v;
22            len[v] = len[u] + 2;
23            cnt[v] = cnt[fail[v]] + 1;
24        }
25        last = tr[u][c - 'a'];
26    }
27    int get_fail(int u, int i) {
28        while (i - len[u] - 1 <= -1 || s[i - len[u] - 1] != s[i]) {
29            u = fail[u];
30        }
31        return u;
32    }
33 };

```

1.10 后缀自动机 (SAM)

字符串 S 的 SAM 是一个接受 S 的所有后缀的最小 DFA。

对于一个长度为 N 的字符串, 它的空间复杂度仅为 $\mathcal{O}(N)$ 。而且, 构造 SAM 的时间复杂度也仅为 $\mathcal{O}(N)$ 。准确地说, 一个 SAM 最多有 $2N - 1$ 个结点和 $3N - 4$ 条转移边。最终得到的是一个 DAG。

所有状态和所有后缀链接构成根为 t_0 的根向树, 这个树也称为后缀链接树。

后缀链接树有如下性质:

- 祖先节点对应的字符串总是子孙节点对应的字符串的后缀。
- 每个节点处的 `endpos` 集合就是它的子树内的所有「前缀节点」 v_i 的下标 i 的集合。
- 后缀链接树的祖先节点的 `endpos` 集合总是严格包含子孙节点的 `endpos` 集合。
- 每个节点处的 `len` 的值就是它的子树内的所有「前缀节点」 v_i 对应前缀的最长公共后缀的长度。
- 除根节点 t_0 外, 每个节点对应的不同子串的数目, 就是它的 `len` 值, 减去它的父节点的 `len` 值, 即 $\text{len}(v) - \text{len}(\text{link}(v))$ 。

```

1 template<int N = 26>
2 struct SAM{
3     struct State{
4         int fa; // 后缀链接 (suffix link)
5         int len; // 该状态代表的最长子串长度
6         std::array<int, N> next; // 转移边
7         State(): fa(0), len(0) {next.fill(0);}
8     };
9
10    std::vector<State> st;
11    std::vector<int> occ; // 出现次数
12    int last = 1;
13
14    SAM(){
15        init();
16    }
17    SAM(std::string& str){
18        init();
19        for(auto ch : str){
20            extend(ch);
21        }
22    }

```

```

23
24 void init(){
25     st.resize(2);
26     occ.assign(2,0);
27     st[0].len = -1;
28     st[1].len = st[1].fa = 0;
29 }
30
31 void extend(char ch){
32     int c = ch - 'a';
33     int cur = st.size(), p = last;
34     st.push_back(State());
35     occ.push_back(1);
36     st[cur].len = st[last].len + 1;
37
38     while(p && !st[p].next[c]){
39         st[p].next[c] = cur;
40         p = st[p].fa;
41     }
42
43     if(!p){
44         st[cur].fa = 1;
45     }
46     else{
47         int q = st[p].next[c];
48         if(st[p].len + 1 == st[q].len){
49             st[cur].fa = q;
50         }
51         else{
52             int clone = st.size();
53             st.push_back(st[q]);
54             occ.push_back(0);
55             st[clone].len = st[p].len + 1;
56             while(p && st[p].next[c] == q){
57                 st[p].next[c] = clone;
58                 p = st[p].fa;
59             }
60             st[q].fa = st[cur].fa = clone;
61         }
62     }
63     last = cur;
64 }
65 };

```

1.10.1 后缀自动机应用

• 检查字符串是否出现

1. **问题：**给定文本串 T 和模式串 P ，判断 P 是否为 T 的子串。
2. **解法：**构造 T 的 SAM。从初始状态 t_0 开始，按 P 的字符转移。若中途失配则 P 未出现；若能完整转移则出现。单次查询 $O(|P|)$ ，且可求得 P 在

T 中的最长匹配前缀。

• 计算不同子串个数

1. **问题：**给定字符串 S ，求其不同子串的总数。
2. **解法：**构造 S 的 SAM。每个状态 v （不含 t_0 ）贡献 $\text{len}(v) - \text{len}(\text{link}(v))$ ，累加所有状态即为答案。复杂度 $O(|S|)$ 。

• 所有不同子串的总长度

1. **问题：**给定字符串 S ，求所有不同子串的长度之和。
2. **解法：**状态 v 贡献 $\frac{\text{len}(v)(\text{len}(v) + 1)}{2} - \frac{\text{len}(\text{link}(v))(\text{len}(\text{link}(v)) + 1)}{2}$ ，累加即得总长度。复杂度 $O(|S|)$ 。

• 字典序第 k 大子串

1. **问题：**给定字符串 S ，多组询问，每次查询 S 的所有子串中字典序第 k 大的子串。
2. **解法：**在 SAM 上统计从每个状态出发的路径总数，然后从根开始按字典序贪心查找。预处理 $O(|S|)$ ，单次查询 $O(|\text{ans}| \cdot |\Sigma|)$ 。

• 最小循环移位

1. **问题：**给定字符串 S ，求其字典序最小的循环移位。
2. **解法：**构造 $S + S$ 的 SAM，从根出发贪心走字典序最小的边，直到路径长度为 $|S|$ 。复杂度 $O(|S|)$ 。

• 出现次数（多模式串查询）

1. **问题：**给定文本串 T ，多组询问，每次给模式串 P ，求 P 在 T 中出现的次数。
2. **解法：**构造 T 的 SAM。在 SAM 上，每个非克隆状态初始 $occ = 1$ ，克隆状态 $occ = 0$ 。将状态按 len 降序排序，将 occ 累加到后缀链接指向的父节点。最终 $occ[v]$ 即状态 v 代表的子串出现次数。查询时找到 P 对应状态，输出其 occ 。预处理 $O(|T|)$ ，单次查询 $O(|P|)$ 。

• 第一次出现的位置

1. **问题：**给定文本串 T ，多组查询，每次给模式串 P ，求 P 在 T 中第一次出现的起始位置。

2. **解法：**构造 T 的 SAM。每个非克隆状态 v_i (对应前缀 $T[1..i]$) 初始值为 i , 克隆状态为 ∞ 。按 len 降序排序, 将每个状态的初始值取 $\min$ 传递给后缀链接父节点。最终 $\text{first}[v]$ 为状态 v 代表子串的结束位置最小值。查询时找到 P 对应状态 v , 答案 $\text{first}[v] - |P| + 1$ 。预处理 $O(|T|)$, 单次查询 $O(|P|)$ 。

• 所有出现的位置

1. **问题：**给定文本串 T , 多组查询, 每次给模式串 P , 输出 P 在 T 中的所有出现起始位置。
2. **解法：**找到 P 对应状态 v , 在后缀链接树中遍历 v 的子树, 输出所有非克隆状态对应的前缀位置 (减去 $|P| + 1$)。单次查询复杂度 $O(|P| + \text{answer}(P))$ 。

• 最短的未出现字符串

1. **问题：**给定字符串 S 和字符集, 求最短的未在 S 中出现的字符串。
2. **解法：**在 S 的 SAM 上 DP。设 d_v 为从状态 v 出发需添加的最小字符数。若 v 无任何转移, 则 $d_v = 1$; 否则 $d_v = 1 + \min_c d_{\text{next}(v,c)}$ 。答案 d_{t_0} , 并可回溯构造。复杂度 $O(|S| \cdot |\Sigma|)$ 。

• 两个字符串的最长公共子串

1. **问题：**给定字符串 S 和 T , 求它们的最长公共子串。
2. **解法：**构造 S 的 SAM。遍历 T , 维护当前匹配状态 v 和长度 l : 若存在 c 转移则转移并 $l + 1$; 否则沿后缀链接回退, 直到有转移或回到根。更新答案 $\text{ans} = \max(\text{ans}, l)$ 。复杂度 $O(|T|)$ 。

• 多个字符串的最长公共子串

1. **问题：**给定 k 个字符串 $S_1, \dots, S_k$, 求它们的最长公共子串。
2. **解法：**构造新串 $T = S_1 \#_1 S_2 \#_2 \cdots S_k \#_k$, 其中 $\#_i$ 为互不相同的分隔符。构造 T 的 SAM, 对每个状态记录它能通过哪些分隔符 (即它能到达哪些原始字符串)。通过 DP 或 DFS 计算每个状态的可达分隔符集合, 若可达全部 k 个, 则该状态对应的最长子串长度为 $\text{len}(v)$ 。取最大值。复杂度 $O(|T| + k \cdot |\Sigma|)$ 。